AquaCanvas: Real Time Watercolor Simulation

CS 184 Final Project Report, Spring 2026

Team Dubuworld

Daeyoung Kim, Jaegyeom Kim, Brian Sui, Shenrui Wu

Final Webpage Final Presentation Final Video Proposal Milestone

Abstract

AquaCanvas is a real time watercolor painting tool built in Godot 4.4.1 that simulates physically based fluid dynamics entirely on the GPU. The simulation runs five compute shader stages (evaporation, force calculation, water and pigment transport, deposition, and diffusion) each frame on a 256×256 canvas at interactive frame rates. Starting from a CPU prototype that could only run at 64×64, we ported the entire physics pipeline to GLSL compute shaders via Godot's RenderingDevice API. The final tool supports pressure sensitive brushes (Wacom tablet), canvas tilt, real time physics parameter adjustment, and PNG export. The result is a painting tool that produces recognizable watercolor effects like wet on wet bleeding, edge darkening, and gravity driven dripping, all of which emerge naturally from the physics rather than from textures or tricks.

Technical Approach

Starting Point

At the milestone we had a CPU based simulation running at 64×64. All five physics stages were working but everything ran in GDScript on the CPU, which was too slow for larger canvases. The milestone was the reference we validated the physics against before moving to GPU.

GPU Physics Pipeline

We rewrote the entire simulation as GLSL 450 compute shaders dispatched through Godot's `RenderingDevice` API. Each shader runs in 8×8 work groups over the full 256×256 canvas. All textures use `R32G32B32A32_SFLOAT` format and are double buffered (shaders read from one copy and write to the other) to avoid read/write conflicts. The five stages run in order every frame:

- **Evaporation.** Each cell loses water proportional to its exposed surface area and a tunable evaporation constant.
- **Force calculation.** A 4-directional displacement map accumulates gravity, surface tension (looks ahead multiple pixels, stops at dry boundaries), and spreading force from immediate neighbors. Force cancellation between opposing directions prevents oscillation.
- **Water and pigment transport.** Water moves according to the displacement map and carries mobile pigment proportionally.
- **Deposition.** Mobile pigment settles onto the static (dried) layer scaled by local water amount and a paper absorbency map (randomized 0.1–0.2).
- **Diffusion.** Static pigment spreads slightly into neighboring dry areas to soften edges.

We deviated from the David Small formulation for water redistribution. The paper's scheme produced a checkerboard artifact at our grid scale, so we switched to a 4-directional displacement approach where each cell computes its outflow to each neighbor independently before any values are updated.

Brush System

Brush strokes are applied via an `add_paint.glsl` shader that uploads a circular dab at the stroke position each frame. A per stroke pixel mask prevents repainting the same pixel twice within one stroke, eliminating the opacity doubling artifact that appears with naive blending. The removing brush uses Beer-Lambert mass conversion (`alpha = 1 - exp(-K * mass)`) so removal is consistent across alpha values and works on the mobile layer first before touching the dried static layer.

Problems Encountered

- **Checkerboard artifact.** The original Small paper redistribution scheme caused a flickering checkerboard at our resolution. Because each cell both read from and wrote to the same buffer in the same pass, neighboring cells saw each other's partially updated values, producing alternating high/low water patterns. We resolved this by precomputing a displacement map in one pass and applying it in a second pass, so reads and writes never mix in the same frame.

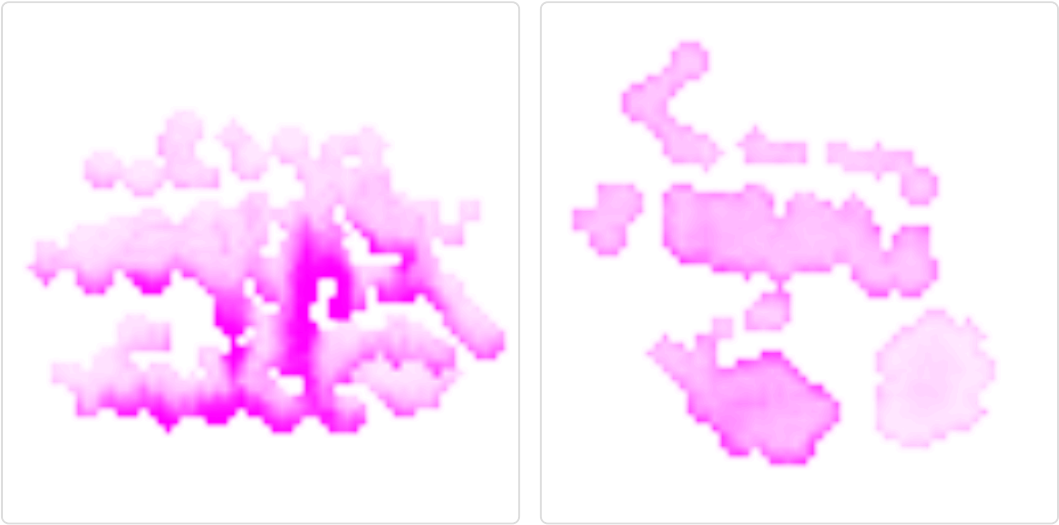
- **Outflow model to inflow model.** Our first water transport shader was outflow based: each cell pushed water to its neighbors according to the displacement map. This caused conservation errors because multiple cells could simultaneously push into the same neighbor, summing to more water than physically possible. We switched to an inflow model where each cell pulls from its neighbors instead, making it straightforward to clamp the total incoming water to the available supply and preserve mass exactly.
- **CPU to GPU stroke latency.** When we first ported brush strokes to GPU, we were still computing the stroke dab position and radius on the CPU each frame and uploading a uniform buffer before dispatching the compute shader. Under high input rates (fast Wacom strokes) the round trip stalled the render thread and produced visible gaps between dabs. We resolved this by batching all stroke events that arrived within one frame into a single buffer upload, dispatching the `add_paint.glsl` shader once per event in sequence rather than waiting for the next frame, which eliminated the gaps without increasing per frame GPU cost.

Lessons Learned

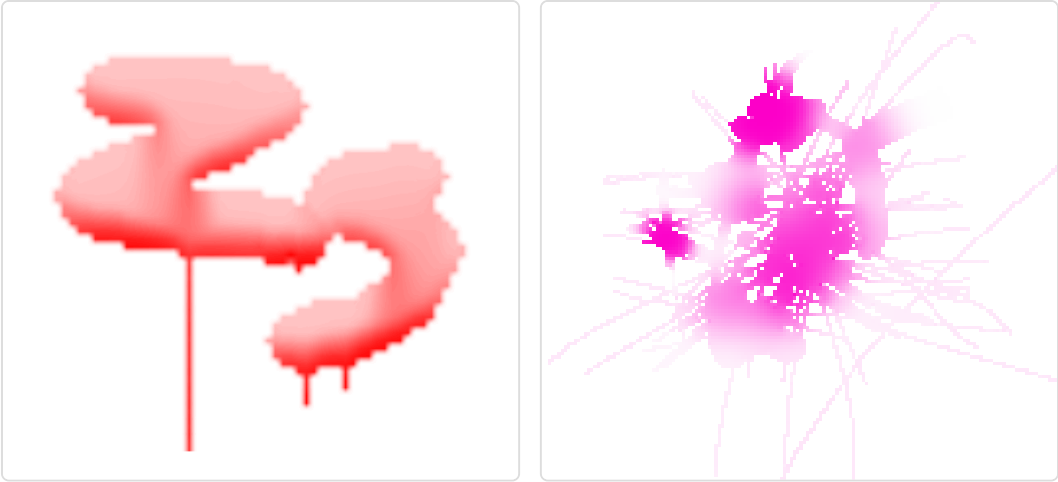
- Double buffering is non-negotiable for stable grid based fluid sims. Single buffer updates cause visible ordering artifacts immediately.
- GPU APIs (even through a game engine) have undocumented state requirements. `texture_update()` and texture creation are not equivalent operations even when the data is the same.
- Start with a CPU reference. Having a ground truth made debugging the GPU port much faster; when output diverged we could compare pixel by pixel.
- Physics that look correct at small scale do not always look correct at large scale. Several parameter values needed retuning after moving from 64×64 to 256×256.

Results

Final demo video (replace embed ID with final video when uploaded).



Sample paintings showing wet on wet bleeding and edge darkening.



Left: gravity driven drip from canvas tilt. Right: pigment diffusion effect over time.

References

- Curtis, C. J., Anderson, S. E., Seims, J. E., Fleischer, K. W., and Salesin, D. H. *Computer-Generated Watercolor*. SIGGRAPH 1997.
- Small, D. *Modeling Watercolor by Simulating Diffusion, Pigment, and Paper Fibers*.
- Chu, N. and Tai, C.-L. *MoXi: Real-Time Ink Dispersion in Absorbent Paper*. SIGGRAPH 2005.
- Van Laerhoven, T. and Van Reeth, F. *Real-Time Simulation of Watery Paint*. Computer Animation and Virtual Worlds, 2005.
- Godot Engine 4.4.1 RenderingDevice API documentation.

Contributions

Member	Contributions
Daeyoung Kim	Overall project lead. Drove architecture decisions and contributed to every part of the system. Produced demo videos. Participated in weekly algorithm discussion meetings and parameter tuning experiments.
Jaegyeom Kim	Contributed to fluid simulation logic and helped resolve the checkerboard artifact. Edited both the milestone and final project videos. Participated in weekly algorithm discussion meetings and parameter tuning experiments.

Brian Sui	Assisted in GPU pipeline debugging. Proposed and implemented the CPU-side input buffer to resolve stroke batching latency. Participated in weekly algorithm discussion meetings and parameter tuning experiments.
Shenrui Wu	Contributed to the user interface and color representation, including the Beer-Lambert opacity model for the eraser brush. Participated in weekly algorithm discussion meetings and parameter tuning experiments.